Lab 06

Invoking the API

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **`<GW_HTTPS_PORT>` / `<GW_HTTP_PORT>`** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue. The -k option in the curl commands skips the same check.

Training Objective

Learn how to test an API from the Developer Portal and with cURL, and how to deploy and test the PizzaShack web application, which gets tokens with the `order_pizza` scope on behalf of its users.

Business Scenario

After subscribing to the API, partners access it through the PizzaShack web application. The application uses the API Manager token endpoint to get JWT access tokens on demand, and only users allowed to order get the `order_pizza` scope.

High-Level Steps

- Test the API using the Developer Portal
- Test the API using cURL
- Deploy and test the PizzaShack application

Part 1 – Test the API from the Developer Portal

1. Log in to the Developer Portal as the **subscriber** user:

   ```
   https://<APIM_HOST>:<HTTPS_PORT>/devportal
   ```

   Example:

   ```
   https://localhost:9443/devportal
   ```

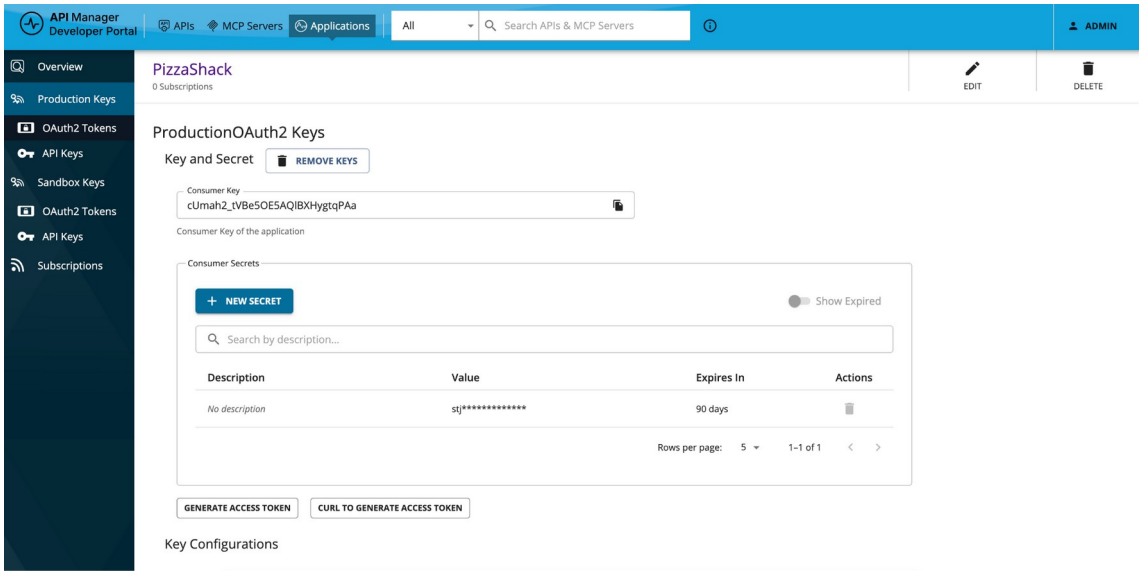
2. Open the **PizzaShack** API.

3. Open the API's **Subscriptions** page. The **PizzaShack** application from the previous lab should be listed. If it is not, subscribe it now (**Unlimited** business plan).

CC BY 4.0

4. If the application has no keys yet, go to **Applications** → **PizzaShack** → **Production Keys** → **OAuth2 Tokens**, select the **Password** and **Client Credentials** grant types, click **Generate Keys** and copy the consumer key and secret (the secret is shown only once).

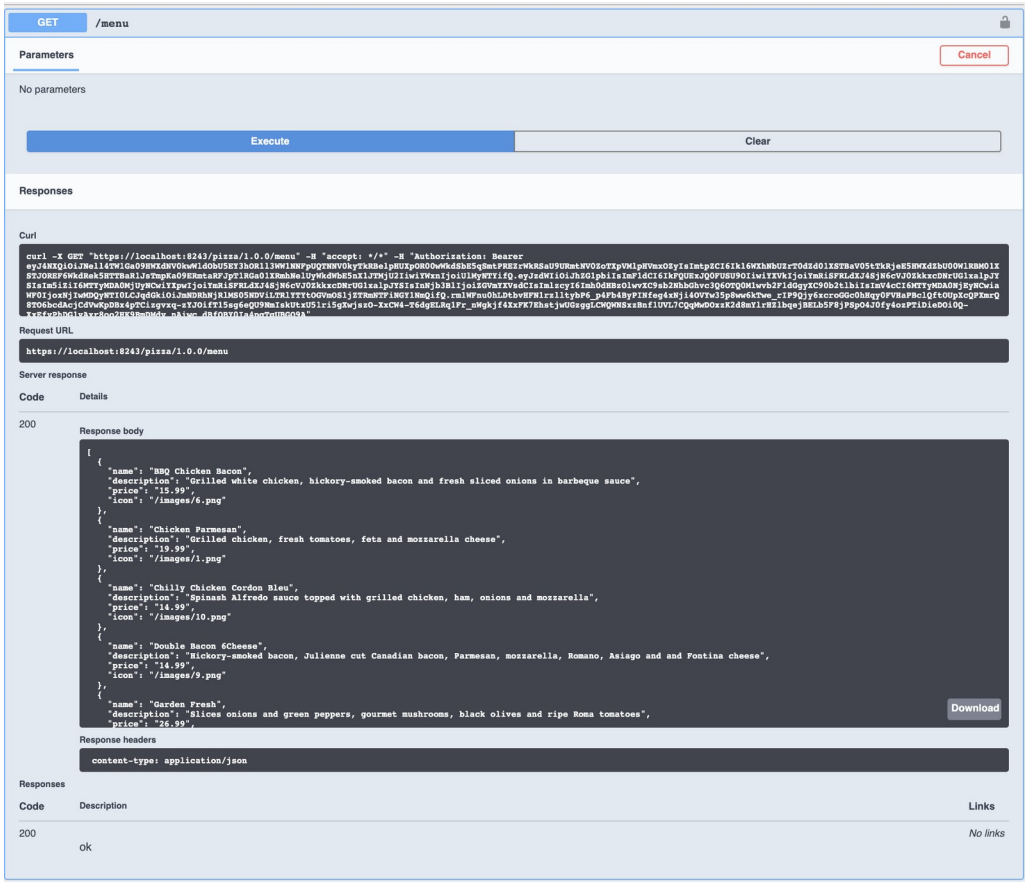
5. Click **Generate Access Token**, paste the consumer secret, select the `order_pizza` scope, click **Generate** and copy the access token.



6. Go back to the API and open **Try Out**.

7. Select the **PizzaShack** application and the **Production** key type. Paste the access token, or click **GET TEST KEY** to generate one.

8. Expand `GET /menu`, click **Try it out** and then **Execute**. The response is HTTP 200 with the list of pizzas.



CC BY 4.0

Part 2 – Test the API using cURL

To call the API with cURL, send the access token in the Authorization header, for example:

```
Authorization: Bearer <ACCESS_TOKEN>
```

Example:

```
Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ
```

Note: You can also use a REST API client of your choice (for example Postman) instead of cURL.

9. Open a new command-line window (Terminal on macOS/Linux, Command Prompt on Windows).

10. Call the menu resource without a token:

```
curl -v http://<APIM_HOST>:<GW_HTTP_PORT>/pizzashack/1.0.0/menu
```

Example:

```
curl -v http://localhost:8280/pizzashack/1.0.0/menu
```

Or, in a REST client:

```
Method: GET
URL:    http://<APIM_HOST>:<GW_HTTP_PORT>/pizzashack/1.0.0/menu
```

Example:

```
Method: GET
URL:    http://localhost:8280/pizzashack/1.0.0/menu
```

The gateway rejects the call because there is no credential (output shortened):

```
> GET /pizzashack/1.0.0/menu HTTP/1.1
> Host: <APIM_HOST>:<GW_HTTP_PORT>
< HTTP/1.1 401 Unauthorized
{"code":"900902","message":"Missing Credentials",
 "description":"Invalid Credentials. Make sure your API invocation call has a header:
 'Authorization : Bearer ACCESS_TOKEN' or 'Authorization : Basic ACCESS_TOKEN'
 or 'ApiKey : API_KEY'"}
```

11. Call it again with the access token you generated with the `order_pizza` scope. This example uses the HTTPS gateway port; `-k` accepts the self-signed certificate:

macOS / Linux

```
curl -k \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

Example – macOS / Linux:

```
curl -k \
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
  https://localhost:8243/pizzashack/1.0.0/menu
```

CC BY 4.0

Windows (Command Prompt)

```
curl -k ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

Example – Windows (Command Prompt):

```
curl -k ^
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" ^
  https://localhost:8243/pizzashack/1.0.0/menu
```

`<ACCESS_TOKEN>` is the token generated for the PizzaShack application. The response is HTTP 200 with the menu (shortened):

```
[{"name":"BBQ Chicken Bacon",
  "description":"Grilled white chicken, hickory-smoked bacon and fresh sliced
                 onions in barbeque sauce",
  "price":"...","icon":"/images/6.png"},
 {"name":"Chicken Parmesan", ...}, ...]
```

12. Place an order. `POST /order` is protected by the `order_pizza` scope. First save the order below as a file named `order.json` in the folder where you run curl:

```
{
  "customerName": "Amina",
  "pizzaType": "Chicken Parmesan",
  "quantity": 2,
  "address": "Nairobi",
  "creditCardNumber": "4111111111111111",
  "delivered": false
}
```

Then send it:

macOS / Linux

```
curl -k -X POST \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" \
  -d @order.json \
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/order
```

Example – macOS / Linux:

```
curl -k -X POST \
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
  -H "Content-Type: application/json" \
  -d @order.json \
  https://localhost:8243/pizzashack/1.0.0/order
```

Windows (Command Prompt)

```
curl -k -X POST ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  -H "Content-Type: application/json" ^
  -d @order.json ^
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/order
```

CC BY 4.0

Example – Windows (Command Prompt):

```
curl -k -X POST ^
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" ^
  -H "Content-Type: application/json" ^
  -d @order.json ^
  https://localhost:8243/pizzashack/1.0.0/order
```

The response is HTTP 201 with the new order, including its `orderId`. If you repeat the call with a token that does not have the `order_pizza` scope, the gateway returns HTTP 403 with code 900910 (Scope validation failed). 4111111111111111 is a standard test card number, not a real card.

Part 3 – Deploy and Test the PizzaShack Application

Important (API Manager 4.7.0): API Manager blocks requests to web applications that are not on its allow list. Without the change below, every page of the PizzaShack web app returns HTTP 401 (Authorization failure). Do this once, before deploying the web app.

13. Stop API Manager and add the following to `<APIM_HOME>/repository/conf/deployment.toml`:

```
[[resource.access_control]]
context = "(.*)/pizzashack(.*)"
secure = false
http_method = "all"
```

Start API Manager again.

14. Get `pizzashack.war` (provided with the lab files, or download it from the URL below) and copy it to `<APIM_HOME>/repository/deployment/server/webapps`.

```
https://github.com/wso2/WSO2-Training/releases/download/APIM40DFAPIM/pizzashack.war
```

15. After a few seconds the log shows `Deployed webapp: ... [/pizzashack]` and the folder `webapps/pizzashack` is created.

Configure the web application:

16. Open this file in a text editor:

```
<APIM_HOME>/repository/deployment/server/webapps/pizzashack/WEB-INF/web.xml
```

Example:

```
~/wso2am-4.7.0/repository/deployment/server/webapps/pizzashack/WEB-INF/web.xml
```

CC BY 4.0

17. Set `consumerKey` and `consumerSecret` to the PizzaShack application's production keys from the previous lab, and save the file. The application keys must allow the **Password** grant type.

```
<context-param>
    <param-name>consumerKey</param-name>
    <param-value>XXXXXXXXXXXXXXXXXXXXXXXXXXXX</param-value>
</context-param>
<context-param>
    <param-name>consumerSecret</param-name>
    <param-value>YYYYYYYYYYYYYYYYYYYYYYYYYYYY</param-value>
</context-param>
```

The app reloads automatically (log: `Reloaded Context with name: /pizzashack`).

Also check these two values in the same file. The defaults work when the web app runs on the same API Manager with no port offset and the API context is /pizzashack:

Parameter	Default value	When to change
`serverURL`	`http://localhost:8280/pizzashack`	Gateway URL plus API context. Change it if your context is different (for example /pizzashack-jd) or if you use a port offset. The app adds /1.0.0/menu and /1.0.0/order itself.
`loginURL`	`http://localhost:9763/oauth2/token`	Token endpoint: 9763 plus any port offset.

18. Open the web app and log in as **mike** (password: `password`):

```
https://<APIM_HOST>:<HTTPS_PORT>/pizzashack/login.jsp
```

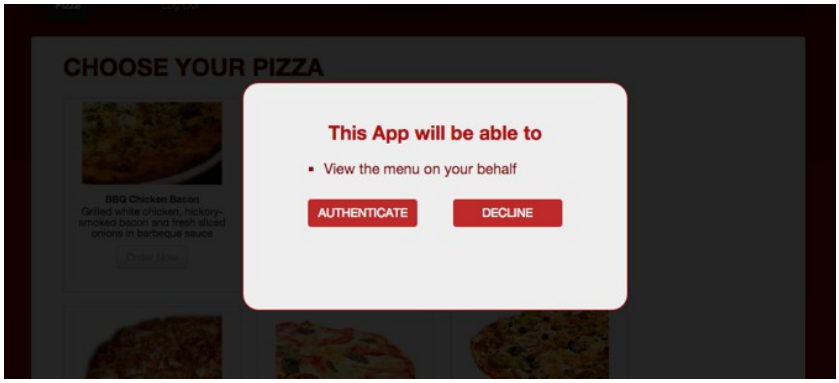
Example:

```
https://localhost:9443/pizzashack/login.jsp
```

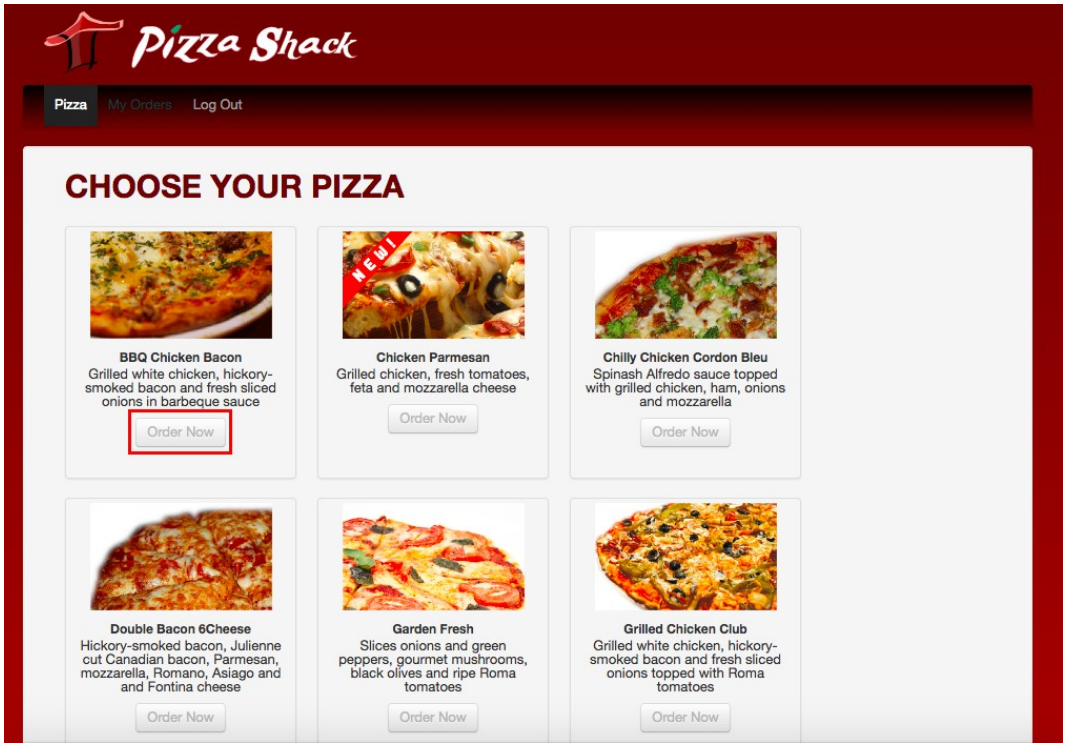


CC BY 4.0

The web app asks for a token with the `order_pizza` scope. Only users with the **admin** or **Internal/subscriber** role can get that scope, and mike has neither, so mike's token does not include it. The menu is shown, but ordering is disabled. If an order were sent with this token, the gateway would reject it with HTTP 403.

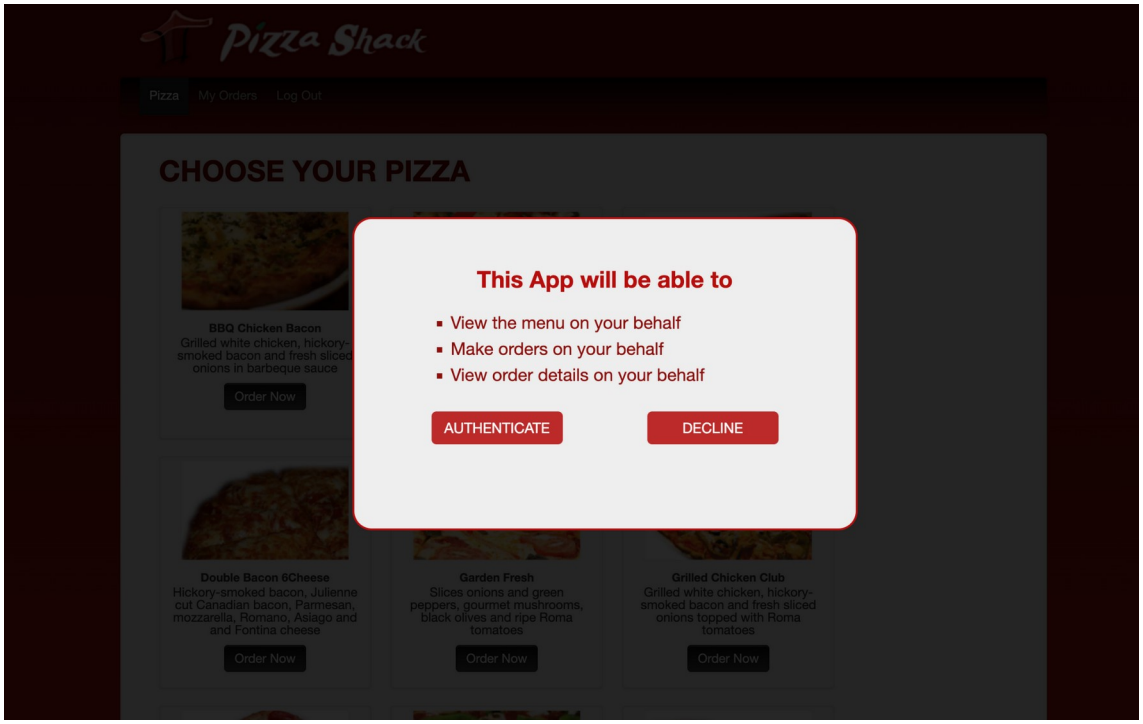


Note that the **Order Now** button is disabled in the image below.

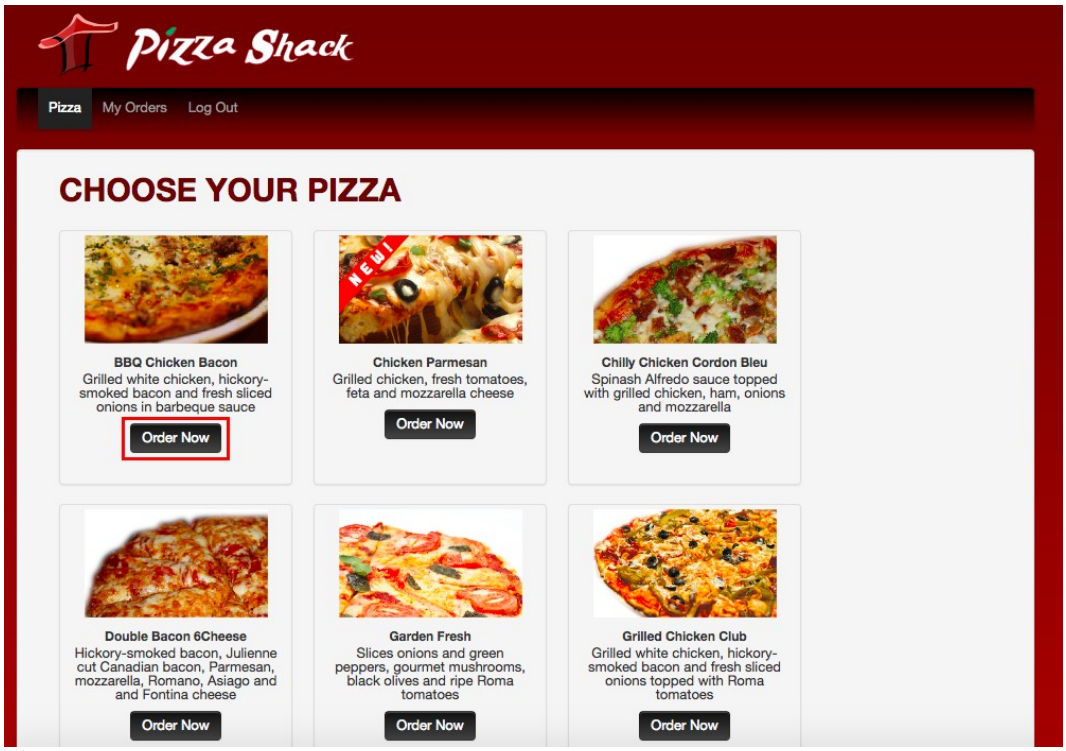


CC BY 4.0

19. Log out and log in as **subscriber**. This user has the **Internal/subscriber** role, so the token includes the `order_pizza` scope and the `/order` resource of the PizzaShack API can be invoked. You see the screen below.



Ordering is now enabled. Click **Order Now** for a pizza, fill in the form and submit it: the order is created through the API (`POST /order`) and its order ID is shown.



Expected Outcome

The API was tested from the Developer Portal and with cURL (no token: 401; token without `order_pizza` on POST /order: 403; valid token: 200 and 201). The PizzaShack web application was deployed in API Manager and tested with two users: mike cannot order, and subscriber can.

CC BY 4.0